

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по Лабораторной работе №4

по дисциплине «Практика и Реализация Баз Данных»

Автор: Конанчук Иван Алексеевич

Факультет: ФПИИ

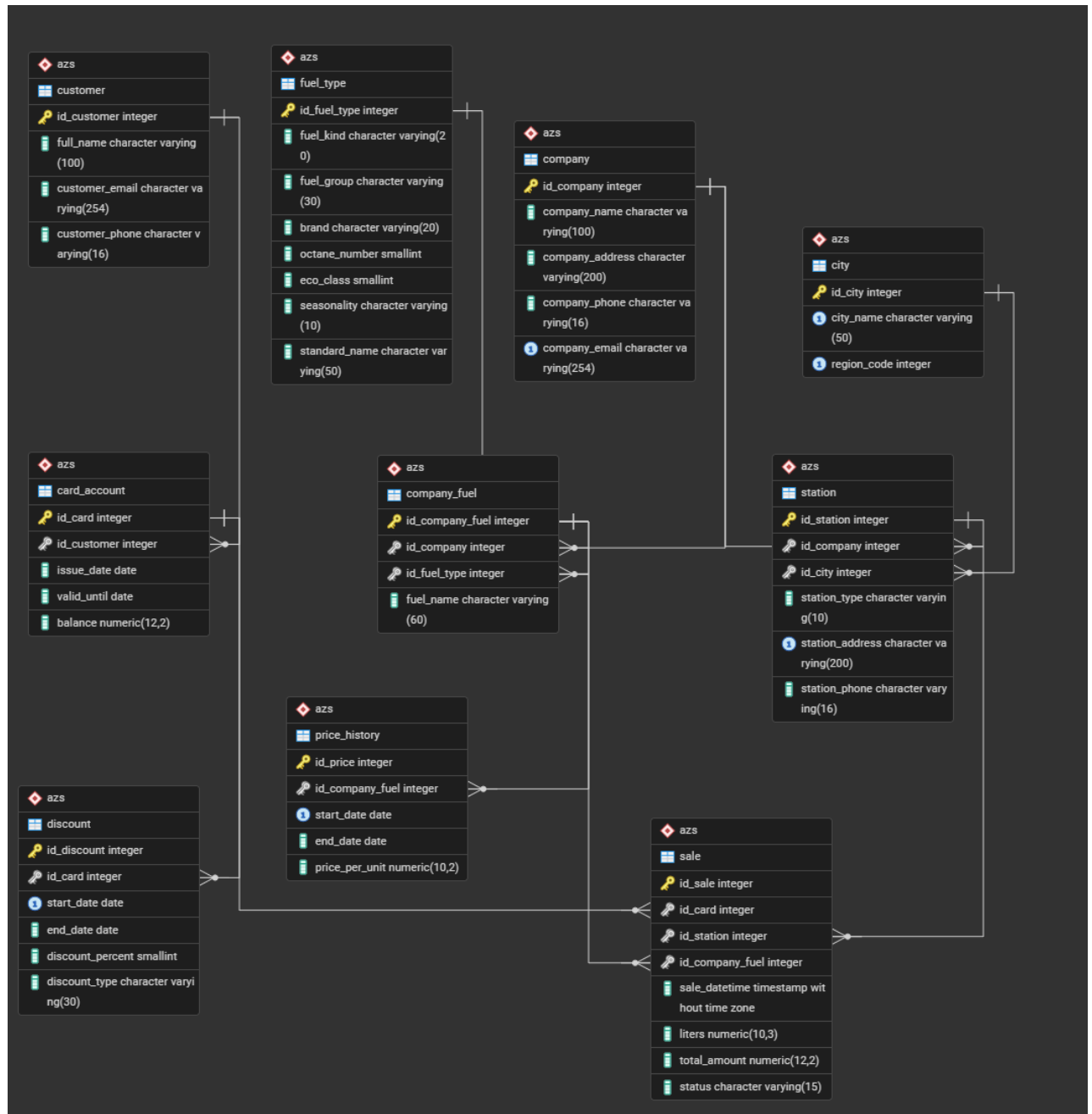
Группа: К3240



Цель работы:

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

1. Схема базы данных



2. Запросы на выборку данных

Запрос 1: Вывести список продаж с указанием клиента, карты, станции, города, фирмы, топлива, количества литров и итоговой стоимости.

```
Query  Query History
1  set search_path to azs, public;
2
3  select
4      s.id_sale,
5      s.sale_datetime,
6      c.full_name as customer_name,
7      ca.id_card,
8      co.company_name,
9      ci.city_name,
10     st.station_address,
11     cf.fuel_name,
12     ft.brand,
13     s.liters,
14     s.total_amount,
15     s.status
16 from sale s
17 join card_account ca on s.id_card = ca.id_card
18 join customer c on ca.id_customer = c.id_customer
19 join station st on s.id_station = st.id_station
20 join city ci on st.id_city = ci.id_city
21 join company co on st.id_company = co.id_company
22 join company_fuel cf on s.id_company_fuel = cf.id_company_fuel
23 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
24 order by s.sale_datetime;
```

Выдаёт правильно

Data Output Messages Notifications									
Showing rows: 1 to 4 Page No: 1 of 1									
	id_sale	sale_datetime	customer_name	id_card	company_name	city_name	station_address	fuel_name	brand
1	integer	timestamp without time zone	character varying (100)	integer	character varying (100)	character varying (50)	character varying (200)	character varying (60)	character varying
1	1	2026-03-05 10:15:00	Ivan Ivanov	1	Lukoil	Saint Petersburg	Saint Petersburg, Nevsky Ave, 10	AI-92 Lukoil	AI-92
2	2	2026-03-06 12:40:00	Petr Petrov	2	Lukoil	Moscow	Moscow, Leningradsky Ave, 15	AI-95 Lukoil	AI-95
3	3	2026-03-07 09:20:00	Anna Sidorova	3	Gazprom Neft	Kazan	Kazan, Pobedy Ave, 40	DT Gazprom Neft	DT
4	4	2026-03-08 18:05:00	Ivan Ivanov	1	Gazprom Neft	Saint Petersburg	Saint Petersburg, Moskovsky Ave, ...	AI-95 G-Drive	AI-95

Запрос 2: Посчитать суммарную выручку и общий объем проданного топлива по каждой фирме.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

```
set search_path to azs, public;

select
    co.company_name,
    count(s.id_sale) as sales_count,
    sum(s.liters) as total_liters,
    sum(s.total_amount) as total_revenue
from sale s
join station st on s.id_station = st.id_station
join company co on st.id_company = co.id_company
where s.status = 'completed'
group by co.company_name
order by total_revenue desc;
```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	company_name character varying (100)	sales_count bigint	total_liters numeric	total_revenue numeric
1	Gazprom Neft	2	50.250	3254.77
2	Lukoil	2	50.500	2819.69

Запрос 3: Вывести текущие цены на топливо по фирмам.

QueryQuery History

Scratch

```
1 set search_path to azs, public;
2
3 select
4     co.company_name,
5     ft.fuel_kind,
6     ft.brand,
7     cf.fuel_name,
8     ph.price_per_unit,
9     ph.start_date,
10    ph.end_date
11 from price_history ph
12 join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
13 join company co on cf.id_company = co.id_company
14 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
15 where current_date between ph.start_date and coalesce(ph.end_date, date '9999-12-31')
16 order by co.company_name, ft.brand;
```

Data OutputMessagesNotifications

Showing rows: 1 to 4

	company_name character varying (100)	fuel_kind character varying (20)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date	end_date date
1	Gazprom Neft	gasoline	AI-95	AI-95 G-Drive	63.20	2026-03-01	[null]
2	Gazprom Neft	diesel	DT	DT Gazprom Neft	68.90	2026-03-01	[null]
3	Lukoil	gasoline	AI-92	AI-92 Lukoil	57.90	2026-03-01	[null]
4	Lukoil	gasoline	AI-95	AI-95 Lukoil	61.40	2026-03-01	[null]

Запрос 4: Найти клиентов, сумма покупок которых выше средней суммы покупок по клиентам.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

```
set search_path to azs, public;

with customer_totals as (
  select
    c.id_customer,
    c.full_name,
    sum(s.total_amount) as customer_total
  from customer c
  join card_account ca on c.id_customer = ca.id_customer
  join sale s on ca.id_card = s.id_card
  where s.status = 'completed'
  group by c.id_customer, c.full_name
)
select
  full_name,
  customer_total
from customer_totals
where customer_total > (
  select avg(customer_total)
  from customer_totals
)
order by customer_total desc;
```

Data Output

Messages

Notifications

≡+

SQL

	full_name character varying (100)	customer_total numeric
1	Ivan Ivanov	2593.26
2	Anna Sidorova	2339.16

Запрос 5: Посчитать количество продаж, объем проданного топлива и выручку по городам.

Query

Query History

↗

Scratch

1

2

3

4

5

6

7

8

9

10

11

12

13

```
set search_path to azs, public;

select
    ci.city_name,
    count(s.id_sale) as sales_count,
    sum(s.liters) as total_liters,
    sum(s.total_amount) as total_revenue
from sale s
join station st on s.id_station = st.id_station
join city ci on st.id_city = ci.id_city
where s.status = 'completed'
group by ci.city_name
order by total_revenue desc;
```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

📥

📥

📈

SQL

Showing rows: 1 to 3

✎

🖨️

 Page No:

1

 of 1

⏪

⏴

⏵

⏩

	city_name character varying (50) 🔒	sales_count bigint 🔒	total_liters numeric 🔒	total_revenue numeric 🔒
1	Saint Petersburg	2	45.750	2593.26
2	Kazan	1	35.000	2339.16
3	Moscow	1	20.000	1142.04

Запрос 6: Вывести карты клиентов, по которым сейчас действует скидка.

Query

Query History

```

1  set search_path to azs, public;
2
3  select
4      ca.id_card,
5      c.full_name,
6      d.discount_percent,
7      d.discount_type,
8      d.start_date,
9      d.end_date
10 from discount d
11 join card_account ca on d.id_card = ca.id_card
12 join customer c on ca.id_customer = c.id_customer
13 where current_date between d.start_date and coalesce(d.end_date, date '9999-12-31')
14 order by d.discount_percent desc;

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📤

📥

📶

SQL

	id_card integer	full_name character varying (100)	discount_percent smallint	discount_type character varying (30)	start_date date	end_date date
1	2	Petr Petrov	7	personal	2026-02-01	[null]
2	1	Ivan Ivanov	5	permanent	2026-01-01	[null]
3	3	Anna Sidorova	3	promo	2026-03-01	2026-12-31

3. Представления

Представление 1: Создать представление с полной детализацией продаж: клиент, карта, фирма, город, станция, топливо, цена, скидка и итоговая стоимость.

Создаем:


```
Query  Query History
1  set search_path to azs, public;
2
3  create or replace view v_sale_details as
4  select
5      s.id_sale,
6      s.sale_datetime,
7      c.full_name as customer_name,
8      ca.id_card,
9      co.company_name,
10     ci.city_name,
11     st.station_address,
12     cf.fuel_name,
13     ft.brand,
14     s.liters,
15     ph.price_per_unit,
16     coalesce(d.discount_percent, 0) as discount_percent,
17     s.total_amount,
18     s.status
19 from sale s
20 join card_account ca on s.id_card = ca.id_card
21 join customer c on ca.id_customer = c.id_customer
22 join station st on s.id_station = st.id_station
23 join city ci on st.id_city = ci.id_city
24 join company co on st.id_company = co.id_company
25 join company_fuel cf on s.id_company_fuel = cf.id_company_fuel
26 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
27 left join price_history ph
28     on ph.id_company_fuel = cf.id_company_fuel
29     and s.sale_datetime::date between ph.start_date and coalesce(ph.end_date, date '9999-12-31')
30 left join discount d
31     on d.id_card = ca.id_card
32     and s.sale_datetime::date between d.start_date and coalesce(d.end_date, date '9999-12-31');
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 302 msec.

Проверяем:

	id_sale integer	sale_datetime timestamp without time zone	customer_name character varying (100)	id_card integer	company_name character varying (100)	city_name character varying (50)	station_address character varying (200)	fuel_name character vary
1	1	2026-03-05 10:15:00	Ivan Ivanov	1	Lukoil	Saint Petersburg	Saint Petersburg, Nevsky Ave, 10	AI-92 Lukoil
2	2	2026-03-06 12:40:00	Petr Petrov	2	Lukoil	Moscow	Moscow, Leningradsky Ave, 15	AI-95 Lukoil
3	3	2026-03-07 09:20:00	Anna Sidorova	3	Gazprom Neft	Kazan	Kazan, Pobedy Ave, 40	DT Gazprom N
4	4	2026-03-08 18:05:00	Ivan Ivanov	1	Gazprom Neft	Saint Petersburg	Saint Petersburg, Moskovsky Ave, ...	AI-95 G-Drive

Представление 2: Создать представление для просмотра актуальных цен на топливо.

Создаем:

```
Query  Query History
1  set search_path to azs, public;
2
3  create or replace view v_current_prices as
4  select
5      co.company_name,
6      ft.fuel_kind,
7      ft.brand,
8      cf.fuel_name,
9      ph.price_per_unit,
10     ph.start_date
11  from price_history ph
12  join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
13  join company co on cf.id_company = co.id_company
14  join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
15  where current_date between ph.start_date and coalesce(ph.end_date, date '9999-12-31');
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 98 msec.

Проверяем:

```
Query  Query History
1  select *
2  from v_current_prices
3  order by company_name, brand;
```

Data Output Messages Notifications

	company_name character varying (100)	fuel_kind character varying (20)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date
1	Gazprom Neft	gasoline	AI-95	AI-95 G-Drive	63.20	2026-03-01
2	Gazprom Neft	diesel	DT	DT Gazprom Neft	68.90	2026-03-01
3	Lukoil	gasoline	AI-92	AI-92 Lukoil	57.90	2026-03-01
4	Lukoil	gasoline	AI-95	AI-95 Lukoil	61.40	2026-03-01

Представление 3: Создать представление со сводной информацией по клиентам: количество покупок, литры и общая сумма.

Создаем:

```
Query  Query History
1  set search_path to azs, public;
2
3  create or replace view v_customer_sales_summary as
4  select
5      c.id_customer,
6      c.full_name,
7      count(s.id_sale) as sales_count,
8      coalesce(sum(s.liters), 0) as total_liters,
9      coalesce(sum(s.total_amount), 0) as total_amount
10 from customer c
11 join card_account ca on c.id_customer = ca.id_customer
12 left join sale s on ca.id_card = s.id_card and s.status = 'completed'
13 group by c.id_customer, c.full_name;
```

Проверяем:

Query Query History Scratch Pad X

1 select *
2 from v_customer_sales_summary
3 order by total_amount desc;

Data Output Messages Notifications

SQL

Showing rows: 1 to 3 of 1

	id_customer integer	full_name character varying (100)	sales_count bigint	total_liters numeric	total_amount numeric
1	1	Ivan Ivanov	2	45.750	2593.26
2	3	Anna Sidorova	1	35.000	2339.16
3	2	Petr Petrov	1	20.000	1142.04

4. Модификация данных с подзапросами

INSERT:

Добавим новую продажу для клиента Ivan Ivanov, топливо AI-95 Lukoil, станция Lukoil в Санкт-Петербурге. ID и связи берутся через подзапросы.

До:

Query

Query History

1

set search_path to azs, public;

2

3

select *

4

from sale

5

order by id_sale;

Data Output

Messages

Notifications

SQL

	id_sale [PK] integer	id_card integer	id_station integer	id_company_fuel integer	sale_datetime timestamp without time zone	liters numeric (10,3)	total_amount numeric (12,2)	status character varying (15)
1	1	1	1	1	2026-03-05 10:15:00	30.500	1677.65	completed
2	2	2	2	2	2026-03-06 12:40:00	20.000	1142.04	completed
3	3	3	4	4	2026-03-07 09:20:00	35.000	2339.16	completed
4	4	1	3	3	2026-03-08 18:05:00	15.250	915.61	completed

Занпок:

set search_path to azs, public;

insert into sale (

id_sale,

id_card,

id_station,

id_company_fuel,

sale_datetime,

liters,

total_amount,

status

)

select

(select coalesce(max(id_sale), 0) + 1 from sale) as id_sale,

(

select ca.id_card

from card_account ca

join customer c on ca.id_customer = c.id_customer

where c.full_name = 'Ivan Ivanov'

limit 1

) as id_card,

(

select st.id_station

from station st

join company co on st.id_company = co.id_company

join city ci on st.id_city = ci.id_city

where co.company_name = 'Lukoil'

and ci.city_name = 'Saint Petersburg'

limit 1

) as id_station,

(

```

select cf.id_company_fuel
from company_fuel cf
join company co on cf.id_company = co.id_company
join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
where co.company_name = 'Lukoil'
      and ft.brand = 'AI-95'
limit 1
) as id_company_fuel,
now() as sale_datetime,
12.500 as liters,
round(
  12.500 * (
    select ph.price_per_unit
    from price_history ph
    join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
    join company co on cf.id_company = co.id_company
    join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
    where co.company_name = 'Lukoil'
          and ft.brand = 'AI-95'
          and current_date between ph.start_date and coalesce(ph.end_date, date
'9999-12-31')
    limit 1
  ) * 0.95,
  2
) as total_amount,
'processing' as status;

```

Проверяем

Query

Query History

1

select *

2

from sale

3

order by id_sale;

Data Output

Messages

Notifications

</

UPDATE

Увеличим актуальную цену топлива AI-95 Lukoil на 2%.

До:

Query

Query History

1

set search_path to azs, public;

2

3

select

4

ph.id_price,

5

co.company_name,

6

ft.brand,

7

cf.fuel_name,

8

ph.price_per_unit,

9

ph.start_date,

10

ph.end_date

11

from price_history ph

12

join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel

13

join company co on cf.id_company = co.id_company

14

join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type

15

where co.company_name = 'Lukoil'

16

and ft.brand = 'AI-95';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id_price integer	company_name character varying (100)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date	end_date date
1	3	Lukoil	AI-95	AI-95 Lukoil	61.40	2026-03-01	[null]

Запрос:

Query	Query History
1	set search_path to azs, public;
2	
3	update price_history
4	set price_per_unit = round(price_per_unit * 1.02, 2)
5	where id_company_fuel in (
6	select cf.id_company_fuel
7	from company_fuel cf
8	join company co on cf.id_company = co.id_company
9	join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
10	where co.company_name = 'Lukoil'
11	and ft.brand = 'AI-95'
12	)
13	and current_date between start_date and coalesce(end_date, date '9999-12-31');

После:

Query

Query History

1

select

2

ph.id_price,

3

co.company_name,

4

ft.brand,

5

cf.fuel_name,

6

ph.price_per_unit,

7

ph.start_date,

8

ph.end_date

9

from price_history ph

10

join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel

11

join company co on cf.id_company = co.id_company

12

join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type

13

where co.company_name = 'Lukoil'

14

and ft.brand = 'AI-95';

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📄

⬇️

📈

SQL

	id_price integer	company_name character varying (100)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date	end_date date
1	3	Lukoil	AI-95	AI-95 Lukoil	62.63	2026-03-01	[null]

DELETE

Удалить незавершенные продажи клиента Ivan Ivanov. Я специально вставил продажу со статусом processing, теперь ее удалю.

До:

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

```
set search_path to azs, public;

select
    s.id_sale,
    c.full_name,
    s.sale_datetime,
    s.total_amount,
    s.status
from sale s
join card_account ca on s.id_card = ca.id_card
join customer c on ca.id_customer = c.id_customer
where c.full_name = 'Ivan Ivanov'
order by s.id_sale;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📄

⬇️

📈

SQL

	id_sale	full_name	sale_datetime	total_amount	status
1	1	Ivan Ivanov	2026-03-05 10:15:00	1677.65	completed
2	4	Ivan Ivanov	2026-03-08 18:05:00	915.61	completed
3	5	Ivan Ivanov	2026-05-30 00:48:36.57194	729.13	processing

Запрос:

```
Query  Query History
1      set search_path to azs, public;
2
3      delete from sale
4      where id_card in (
5          select ca.id_card
6          from card_account ca
7          join customer c on ca.id_customer = c.id_customer
8          where c.full_name = 'Ivan Ivanov'
9      )
10     and status = 'processing';
```

После:

Query Query History

```
1      select
2          s.id_sale,
3          c.full_name,
4          s.sale_datetime,
5          s.total_amount,
6          s.status
7      from sale s
8      join card_account ca on s.id_card = ca.id_card
9      join customer c on ca.id_customer = c.id_customer
10     where c.full_name = 'Ivan Ivanov'
11     order by s.id_sale;
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 2 of 1

	id_sale integer	full_name character varying (100)	sale_datetime timestamp without time zone	total_amount numeric (12,2)	status character varying (15)
1	1	Ivan Ivanov	2026-03-05 10:15:00	1677.65	completed
2	4	Ivan Ivanov	2026-03-08 18:05:00	915.61	completed

Транзакция удалена!

5. Графическое представление запроса

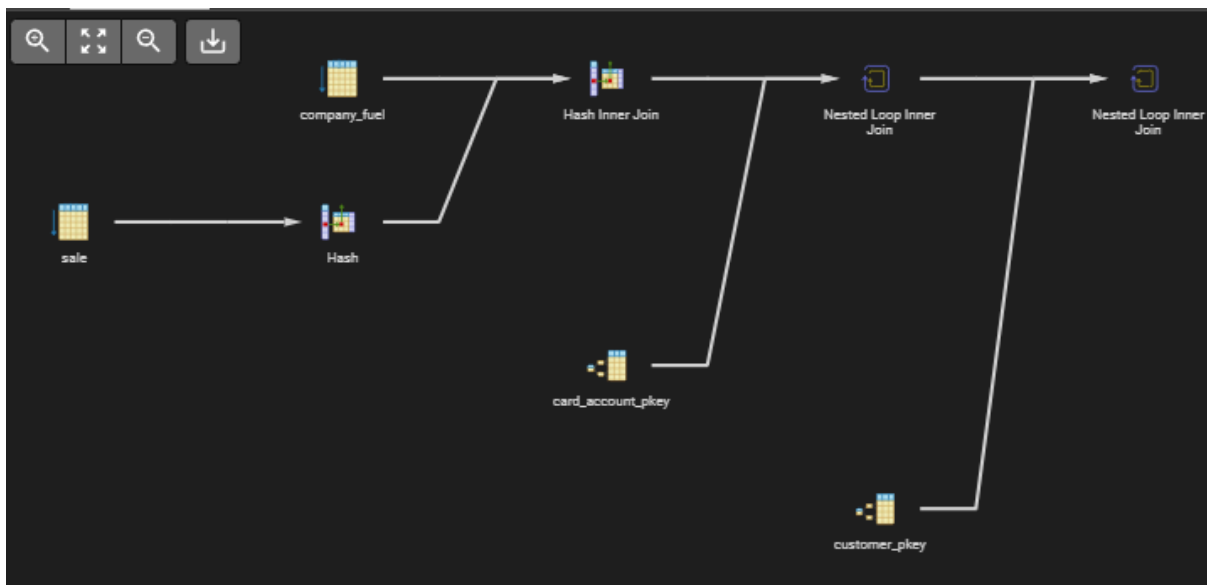
Возьмем запрос по продажам:


```
1  set search_path to azs, public;
2
3  select
4      s.id_sale,
5      s.sale_datetime,
6      c.full_name,
7      cf.fuel_name,
8      s.total_amount
9  from sale s
10 join card_account ca on s.id_card = ca.id_card
11 join customer c on ca.id_customer = c.id_customer
12 join company_fuel cf on s.id_company_fuel = cf.id_company_fuel
13 where s.status = 'completed';
```

Query History

Explain
F7

Открываем графический вид:



7. Query History

Проверим историю выполненных запросов:

Query

Query History

Show queries generated internally by pgAdmin?

Remove

Remove All

Today - 5/30/2026

▶ create schema if not ex...

01:03:28

▶ select s.id_sale, s.sa...

01:00:27

▶ select s.id_sale, s.sa...

01:00:25

E select s.id_sale, s.sa...

01:00:24

▶ select s.id_sale, s.sa...

01:00:19

▶ select s.id_sale, s.sa...

01:00:14

E select s.id_sale, s.sa...

00:58:15

▶ set search_path to azs,...

00:57:14

▶ select s.id_sale, c.ful...

00:55:39

▶ set search_path to azs,...

00:55:17

▶ set search_path to azs

5/30/2026 1 72 msec

1:00:27 AM

Date

Rows

Duration

affected

Copy

Copy to Query Editor

```

select
    s.id_sale,
    s.sale_datetime,
    c.full_name,
    cf.fuel_name,
    s.total_amount
from sale s
join card_account ca on s.i
join customer c on ca.id_cu
join company_fuel cf on s.i
where s.status = 'completed'

```

Messages

Successfully run. Total query runtime: 72 msec. 1 rows affected.

Сейчас тестовых данных у меня мало, поэтому для того чтобы показать работу индексов сгенерируем дополнительных данных:

Query

Query History

```

1  set search_path to azs, public;
2
3  insert into sale (
4      id_sale,
5      id_card,
6      id_station,
7      id_company_fuel,
8      sale_datetime,
9      liters,
10     total_amount,
11     status
12 )
13 select
14     1000 + g as id_sale,
15     ((g - 1) % 3) + 1 as id_card,
16     ((g - 1) % 4) + 1 as id_station,
17     ((g - 1) % 4) + 1 as id_company_fuel,
18     timestamp '2026-04-01 00:00:00' + (g || ' minutes')::interval as sale_datetime,
19     20.000 as liters,
20     1200.00 as total_amount,
21     'completed' as status
22 from generate_series(1, 20000) as g
23 where not exists (
24     select 1
25     from sale s
26     where s.id_sale = 1000 + g
27 );

```

Data Output

Messages

Notifications

INSERT 0 20000

Query returned successfully in 671 msec.

Query

Query History

```

1  select count(*) as sale_count
2  from sale;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

🖨️

Page No: 1

	sale_count
1	20004

Данные заполнились.

Запрос без индекса:

Query

Query History

1

`explain analyze`
`select`
`id_sale,`
`sale_datetime,`
`liters,`
`total_amount`
`from sale`
`where sale_datetime >= timestamp '2026-04-10 00:00:00'`
`and sale_datetime < timestamp '2026-04-11 00:00:00';`

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

⬇️

⚡

SQL

Showing rows: 1 to 8

✎

📧

Page No: 1 of 1

⏪

⏴

⏵

⏩

QUERY PLAN

text

1

Seq Scan on sale (cost=0.00..487.06 rows=1439 width=22) (actual time=0.852..1.483 rows=1439)

2

Filter: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale_datetime < '2026-04-11 00:00:00'::timestamp without time zone))

3

Rows Removed by Filter: 18564

4

Buffers: shared hit=187

5

Planning:

6

Buffers: shared hit=17

7

Planning Time: 0.107 ms

8

Execution Time: 1.533 ms

Время - 1.533 мс.

Создаем простой индекс:

Query

Query History

1

`create index idx_sale_datetime`
`on sale (sale_datetime);`

`analyze sale;`

Запрос с индексом:

Query

Query History

1

explain analyze
 select
 id_sale,
 sale_datetime,
 liters,
 total_amount
 from sale
 where sale_datetime >= timestamp '2026-04-10 00:00:00'
 and sale_datetime < timestamp '2026-04-11 00:00:00';

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 8

✎

🖨️

 Page No: of 1

⏪

⏴

⏵

⏩

	QUERY PLAN
	text
1	Index Scan using idx_sale_datetime on sale (cost=0.29..66.07 rows=1439 width=22) (actual tim
2	Index Cond: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale
3	Index Searches: 1
4	Buffers: shared hit=14 read=6
5	Planning:
6	Buffers: shared hit=27 read=1
7	Planning Time: 1.352 ms
8	Execution Time: 0.354 ms

Время - 0.354 мс уменьшилось почти в 5 раз.

Теперь проверим запрос сразу по станции и дате:

Query

Query History

1

`explain analyze`
`select`
`id_sale,`
`id_station,`
`sale_datetime,`
`total_amount`
`from sale`
`where id_station = 2`
`and sale_datetime >= timestamp '2026-04-10 00:00:00'`
`and sale_datetime < timestamp '2026-04-20 00:00:00'`
`order by sale_datetime;`

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

⤴️

SQL

Showing rows: 1 to 10

✎

🖨️

Page No: 1 of 1

⏪ ⏩ ⏴ ⏵

	QUERY PLAN
	text
1	Index Scan using idx_sale_datetime on sale (cost=0.29..311.69 rows=1760 width=21) (actual time
2	Index Cond: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale_c
3	Filter: (id_station = 2)
4	Rows Removed by Filter: 5281
5	Index Searches: 1
6	Buffers: shared hit=73 read=14
7	Planning:
8	Buffers: shared hit=25 read=1
9	Planning Time: 0.206 ms
10	Execution Time: 1.256 ms

Время - 1.256 мс.

Создаем составной индекс:

```
Query  Query History
1  create index idx_sale_station_datetime
2  on sale (id_station, sale_datetime);
3
4  analyze sale;

Data Output  Messages  Notifications
ANALYZE

Query returned successfully in 80 msec.
```

После проверки получилось такое же время, поэтому добавим еще продаж:

```
Query  Query History
1  set search_path to azs, public;
2
3  insert into sale (
4      id_sale,
5      id_card,
6      id_station,
7      id_company_fuel,
8      sale_datetime,
9      liters,
10     total_amount,
11     status
12 )
13 select
14     (select coalesce(max(id_sale), 0) from sale) + g as id_sale,
15     ((g - 1) % 3) + 1 as id_card,
16     ((g - 1) % 4) + 1 as id_station,
17     ((g - 1) % 4) + 1 as id_company_fuel,
18     timestamp '2026-04-01 00:00:00' + (g || ' minutes')::interval as sale_datetime,
19     20.000 as liters,
20     1200.00 as total_amount,
21     'completed' as status
22 from generate_series(1, 300000) as g;

Data Output  Messages  Notifications
ERROR:  new row for relation "sale" violates check constraint "ck_sale_datetime"
Failing row contains (106042, 1, 2, 2, 2026-05-30 01:22:00, 20.000, 1200.00, completed).

SQL state: 23514
Detail: Failing row contains (106042, 1, 2, 2, 2026-05-30 01:22:00, 20.000, 1200.00, completed).
```

Выдало ошибку, что закономерно, я не учел, что 300000 минут от 2026-04-01 дойдет дальше нынешней даты, и PostgreSQL не пустил её.

Создадим запросы в 2025 году:

Query	Query History
1	<code>set search_path to azs, public;</code>
2	
3	<code>insert into sale (</code>
4	<code>id_sale,</code>
5	<code>id_card,</code>
6	<code>id_station,</code>
7	<code>id_company_fuel,</code>
8	<code>sale_datetime,</code>
9	<code>liters,</code>
10	<code>total_amount,</code>
11	<code>status</code>
12	<code>)</code>
13	<code>select</code>
14	<code>(select coalesce(max(id_sale), 0) from sale) +</code>
15	<code>((g - 1) % 3) + 1 as id_card,</code>
16	<code>((g - 1) % 4) + 1 as id_station,</code>
17	<code>((g - 1) % 4) + 1 as id_company_fuel,</code>
18	<code>timestamp '2025-01-01 00:00:00' + (g ' minut</code>
19	<code>20.000 as liters,</code>
20	<code>1200.00 as total_amount,</code>
21	<code>'completed' as status</code>
22	<code>from generate_series(1, 300000) as g;</code>

Data Output	Messages	Notifications
<code>INSERT 0 300000</code>		
<code>Query returned successfully in 8 secs 434 msec.</code>		

Тестируем без индекса:

Query

Query History

```

1  set search_path to azs, public;
2
3  drop index if exists idx_sale_station_datetime;
4  drop index if exists idx_sale_datetime;
5
6  analyze sale;
7
8  explain analyze
9  select
10     id_sale,
11     id_station,
12     sale_datetime,
13     total_amount
14 from sale
15 where id_station = 2
16     and sale_datetime >= timestamp '2025-04-10 10:00:00'
17     and sale_datetime < timestamp '2025-04-10 11:00:00'
18 order by sale_datetime;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 16

✎

📧

Page No: 1 of 1

⏪

⏴

⏵

⏩

	QUERY PLAN text
9	-> Parallel Seq Scan on sale (cost=0.00..6285.16 rows=10 width=21) (actual time=6.293..11.803 row
10	Filter: ((sale_datetime >= '2025-04-10 10:00:00'::timestamp without time zone) AND (sale_datetim
11	Rows Removed by Filter: 159994
12	Buffers: shared hit=2991
13	Planning:
14	Buffers: shared hit=15
15	Planning Time: 0.121 ms
16	Execution Time: 43.575 ms

И с тем же составным индексом:

Query

Query History

1

```
explain analyze
select
  id_sale,
  id_station,
  sale_datetime,
  total_amount
from sale
where id_station = 2
      and sale_datetime >= timestamp '2025-04-10 10:00:00'
      and sale_datetime < timestamp '2025-04-10 11:00:00'
order by sale_datetime;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 8

✎

🖨

 Page No:

1

 of 1

⏪

⏴

⏵

⏩

QUERY PLAN

text

1

Index Scan using idx_sale_station_datetime on sale (cost=0.42..58.75 rows=14 width=21) (actual rows=8)

2

Index Cond: ((id_station = 2) AND (sale_datetime >= '2025-04-10 10:00:00'::timestamp without time zone AND sale_datetime < '2025-04-10 11:00:00'::timestamp without time zone))

3

Index Searches: 1

4

Buffers: shared hit=2 read=3

5

Planning:

6

Buffers: shared hit=30 read=1

7

Planning Time: 1.326 ms

8

Execution Time: 0.076 ms

✓

Successfully run. Total query runtime: 61 msec. 8 rows.

Теперь видно разницу.

Таблица сравнения:

Запрос	Индекс	Время без индекса	Время с индексом
Поиск продаж по дате	idx_sale_datetime	1.533 ms	0.354 ms
Поиск продаж по станции и дате	idx_sale_station_datetime	43.575 ms	0.076 ms

Удалим индексы:

```

Query  Query History
1      set search_path to azs, public;
2
3      drop index if exists idx_sale_datetime;
4      drop index if exists idx_sale_station_datetime;

```

Сделаем бекап:

Backup (Database: azs_sales_db)

General
Data Options
Query Options
Table Options
Options
Objects

Filenamelab4

FormatPlain

Compression ratio

EncodingSelect an item...

Number of jobs

Role nameSelect an item...

Вывод:

В ходе лабораторной работы были выполнены запросы на выборку данных из базы azs_sales_db. Были составлены запросы с соединением нескольких таблиц, группировкой, агрегатными функциями и подзапросами. Также были созданы представления для детализации продаж, просмотра актуальных цен и анализа покупок клиентов.

Также были выполнены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Для анализа производительности были построены планы выполнения запросов через EXPLAIN ANALYZE, созданы простой и составной индексы, после чего время выполнения запросов было сравнено до и после создания индексов. После проверки индексы были удалены. В результате были закреплены навыки работы с Query Tool, представлениями, планами выполнения запросов и индексами в PostgreSQL.